TMS Service Status Platform Architecture

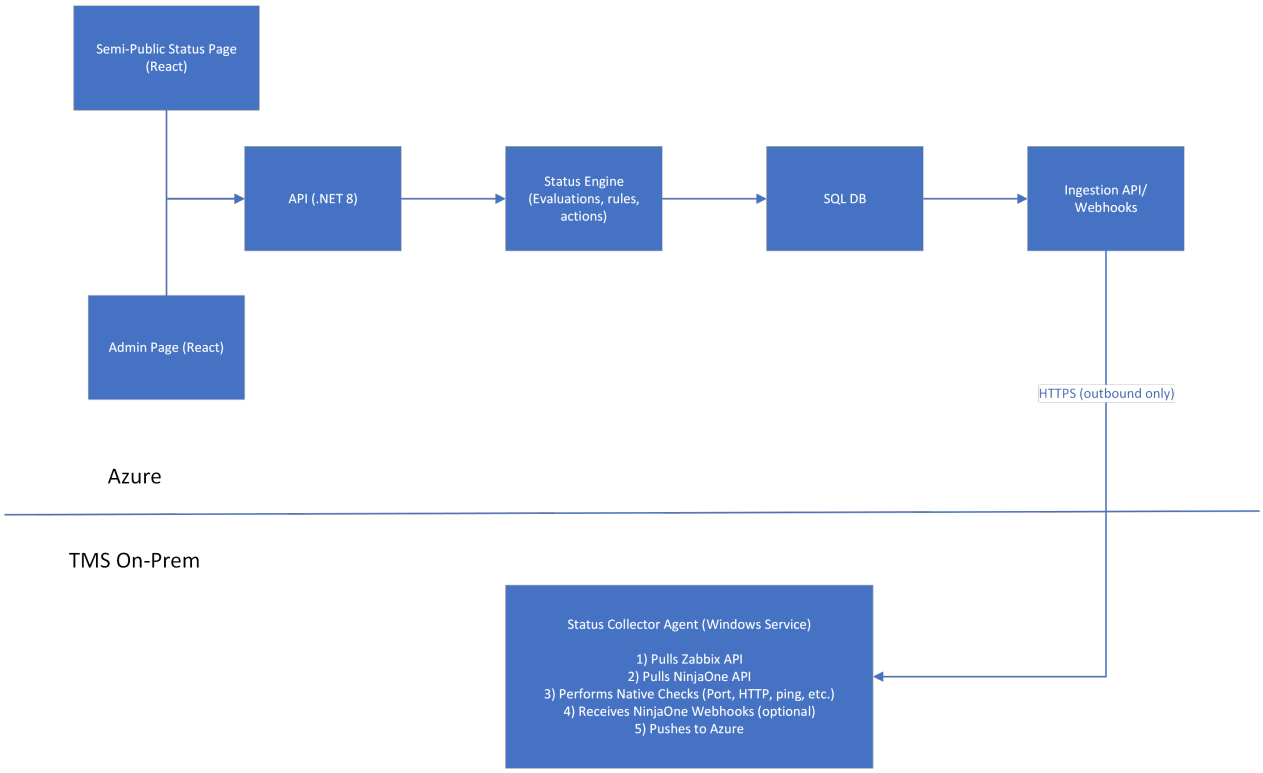
This document describes the architecture, deployment, domain model, integration points (Zabbix, NinjaOne, Native checks), webhook ingestion, and example implementation for TMS' Mail Service.

Objectives

The platform provides:

- A unified service-level health model for IT.
- Combines Zabbix, NinjaOne, and native synthetic checks.
- Always-on status page hosted in Azure.
- Internal admin dashboard with:
  - Manual incidents
  - Overrides
  - Linked NinjaOne tickets
- Optional webhook support from NinjaOne for instant alert ingestion.
- Optional automated ticket creation/closure in NinjaOne.

High Level Architecture



Azure hosts:

- Public/internal status page
- Admin dashboard
- API, Status Engine, DB
- Webhook endpoints
- Automation (NinjaOne ticket creation/closure)

On-prem:

- Lightweight StatusCollector agent close to Zabbix/Ninja/internal systems.
- Outbound HTTPS only; no inbound holes into the client network.

Domain Model

Service

Represents a user-facing business service.

Field	Description
Id	Guid
Name	Display name (e.g. Internal Mail)
Category	e.g. Network, App (optional)
Description	Service description
isUserFacing	Bool
ownerTeam	To be used for tickets/alerts
sortOrder	Ordering on the UI
isActive	Soft deletion flag

Component

Technical building block for a service.

Field	Description
Id	Guid
Name	Display name (e.g. “Exchange2019Cluster”)
Description	Description (optional)
Type	e.g. Cluster, App, LB, DNS (optional)
Criticality	critical | degradedOnly | informational
isExternal	Bool
isActive	Soft deletion flag

ServiceComponentDependency

Defines importance of each component.

Field	Description
Id	Guid
serviceId	Service FK
componentId	Component FK
ImpactType	critical | degradedOnly | informational

critical: outage of this component can cause full service outage.
degradedOnly: outage affects service partially (degraded).
informational: shown to IT, doesn't affect end-user status.

CheckDefinition

Describes a test for a single component.

Field	Description
Id	Guid
ComponentId	Component FK
SourceType	Zabbix | NinjaOne | Native
CheckType	(depends on the above, see below)
ParametersJson	Check configuration
IntervalSeconds	Desired polling interval
TimeoutSeconds	Optional timeout
Enabled	Enabled flag

Supported checkTypes by sourceType:

sourceType = "Zabbix"

- ZabbixTrigger
- ZabbixItem

sourceType = "NinjaOne"

- NinjaDeviceStatus
- NinjaAlert
- NinjaCloudMonitorStatus
- NinjaCloudMonitorAlert

sourceType = "Native"

- HttpSynthetic
- DnsResolution
- TcpPort
- Ping

CheckResult

Produced each time a CheckDefinition runs (or an event is ingested).

Field	Description
Id	Guid
CheckDefinitionId	CheckDefinition FK
ComponentId	Component FK (see note below)
Status	Ok | Warning | Critical | Unknown
Message	Human readable message
NumericValue	Optional metric (latency, CPU%)
Timestamp	Timestamp
DurationMs	How much time the test ran
RawDetailsJson	Raw payload for debug

Note: Duplication of the componentId here is for easily looking up this table and will always be the same as the componentId inside CheckDefinition.

ComponentStatusSnapshot

Represents the current evaluated status of a component at a given time. This will be used for historical/reporting purposes. The Status Engine periodically evaluates ComponentStatus from CheckResults using a rule set.

Field	Description
Id	Guid
ComponentId	ComponentId FK
Status	Operational | Degraded | Outage | Unknown
Reason	Short reason
Timestamp	Timestamp
Source	Auto (from checks) | Manual (override)

ServiceStatusSnapshot

Similar entity as the above, just on the Service level.

Field	Description
Id	Guid
ServiceId	Service FK
Status	Operational | Degraded | Outage | Maintenance | Unknown
Reason	Short reason (will be shown on the UI)
Timestamp	Timestamp
Source	Auto | Manual

Incident

Represents an ongoing or past incident. We will use this in case its not supported by NinjaOne.

Field	Description
Id	Guid
Title	Title
Description	Detailed Description
Severity	Low | Medium | High | Critical
Status	Open | Monitoring | Resolved
StartDate	Start date
ResolutionDate	Resolution date
CreatedBy	System or User
UpdatedBy	Last editor

Relations:

- IncidentService (link table):
  - incidentId → Incident.id
  - serviceId → Service.id
- Optionally links to ExternalTicket via incidentId (see below).

MaintenanceWindow

Planned maintenance affecting one or more services. We will use this in case its not supported by NinjaOne.

Field	Description
Id	Guid
Title	Title
Description	Description
Status	Scheduled | Active | Completed | Cancelled
StartTime	Start time
End Time	End time
CreatedBy	User

Relations:

- **MaintenanceService** (link table):
  - maintenanceId → MaintenanceWindow.id
  - serviceId → Service.id

ExternalTicket

Ticket in NinjaOne.

Field	Description
Id	Guid
ServiceId	Service FK
IncidentId	Incident FK (optional)
ExternalSystem	“NinjaOne”
ExternalTicketId	Id in NinjaOne
Status	Open | Closed
CreatedAt	Time
ClosedAt	Time
URL	Ticket deep link to NinjaOne

Rules

Configure rules for determinging status at the service/component level. E.g.

ServiceRules

Field	Description
ServiceId	ServiceFK
RulesJson	Status rules

Example:

```
{
  "statusEvaluation": {
    "outageIfAnyCriticalComponentOutage": true,
    "degradedIfAnyComponentDegraded": true,
    "ignoreInformational": true
  },
  "actions": {
    "onOutage": {
      "createNinjaTicket": true,
      "priority": "High",
      "assignmentGroup": "Messaging"
    },
    "onRecovery": {
      "closeNinjaTicket": true
    }
  }
}
```

Summary

The service aims to orchestrate and automate incident reporting to both IT and internal personnel by conducting tests which can be either:

- Zabbix Items
- NinjaOne (devices + cloud monitors)
- Native checks (to cover edge cases)
- Webhooks
    - NinjaOne can send us alerts which we will convert to CheckStatus objects

Following the test the status engine computes component + service status according to the rules and:

- Updates the user facing UI
- Automates opening on NinjaOne tickets

TMS IT can also:

- Define maintenance windows
- Manually override computed status

Data from check/component/service statuses is saved to enable analytics and reporting.

Practical Example: Mail Service

In this section we will apply the above architecture to the mail service as per details shared.

Related Services

- EmailExternal
- EmailInternal
- PublicFolders

Related Components

- Exchange2019Cluster
- Exchange2013PFCluster
- KEMPOutlookVS
- KEMPCardiffPFVS
- InternetLinks (3 ISP IPs monitored by Ninja)
- PublicDNSOutlookTms
- InternalDNSCluster
- F5ReverseProxy
- CheckPointCluster
- CoreSwitchLAN

- CiscoFirePower
- ExternalOWASynthetic (optional separate component or check on KEMP)

Each of these has one or more CheckDefinitions attached via componentId.

Example CheckDefinition Entries

Exchange 2019 cluster node health (via Zabbix)

```
{
  "componentId": "Exchange2019Cluster",
  "sourceType": "Zabbix",
  "checkType": "ZabbixTrigger",
  "parametersJson": {
    "host": "tms-exch01.cardiff.local",
    "triggerName": "Exchange 2019 Node Health"
  },
  "intervalSeconds": 60,
  "enabled": true
}
```

Internet link (via NinjaOne)

```
{
  "componentId": "InternetLinks",
  "sourceType": "NinjaOne",
  "checkType": "NinjaDeviceStatus",
  "parametersJson": {
    "deviceId": "ISP1-Device"
  },
  "intervalSeconds": 60,
  "enabled": true
}
```

External OWA synthetic (via Native HTTP test)

```
{
  "componentId": "KEMPOutlookVS",
  "sourceType": "Native",
  "checkType": "HttpSynthetic",
  "parametersJson": {
    "url": "https://outlook.tms-
management.org/owa/healthcheck.html",
    "expectedStatusCode": 200,
    "maxResponseTimeMs": 3000
  },
  "intervalSeconds": 60,
```

```
  "enabled": true
}
```

Component/Service Rules

Exchange 2019 Cluster -> AtLeastNofMHealthy (min 2 out of 3 ok)

Internet Links -> AtLeastNofMHealthy (min 1 out of 3 ok)

KEMPOutlookVS -> WorstCheckStatus

Email External will have all 3 listed in ServiceComponentDependency as critical dependencies. So the conceptual rule will be:

- Outage if:
  - KEMPOutlookVS = outage, OR
  - Exchange2019Cluster = outage, OR
  - InternetLinks = outage
- Degraded if:
  - Any component degraded, OR
  - Exchange2019Cluster = outage (since we can work with 1 our of 3 nodes up, but performance will be degraded)

Furthermore, when Email External goes to outage status, the platform will open a NinjaOne ticket and link it to an Incident. It will also add relevant information to the ticket and assign it to the correct team.